



Maharashtra State Board Technical Education, Mumbai
Government polytechnic, Solapur



Capstone Project-Execution

DIPLOMA IN COMPUTER TECHNOLOGY

Academic Year 2024-25.

A

PROJECT REPORT

ON

**“LexRoBERTa: A Hybrid Extractive Text Summarizer
Combining LexRank and RoBERTa Embeddings”**

Submitted by:

Roll No.	Enrollment No.	Student Name
34	2200150164	Hruthika Chavan
40	2200150172	Shraddha Katkar
41	2200150173	Anushkaa Pawar
93	2200150230	Sonali Kodmur

**Submitted To: -
Prof. R.Y. Gangundi**

Maharashtra State Board of Technical Education



Certificate of Completion Of Capstone Project-Execution

This is to certify that

Roll No.	Enrollment No.	Student Name	Seat No.
34	2200150164	Hruthika Chavan	169241
40	2200150172	Shraddha Katkar	169247
41	2200150173	Anushkaa Pawar	169248
93	2200150230	Sonali Kodmur	169300

Has satisfactorily completed the project planning titled as “LexRoBERTa: A Hybrid Extractive Text Summarizer Combining LexRank and RoBERTa Embeddings” as a partial fulfillment for awarding the diploma in Computer Technology by Maharashtra State Board of Technical Education, Mumbai for the academic year 2024-25.

Project Guide

Head of Department

Principal

Internal Examiner

External Examiner

Date:

Place: Solapur

Acknowledgment

We would like to extend our heartfelt gratitude to all those who have contributed to the successful completion of our project, **"LexRoBERTa: A Hybrid Extractive Text Summarizer Combining LexRank and RoBERTa Embeddings"**

In the accomplishment of this project successfully, many people have best owned upon us their blessings and heart privileged support. Primarily, we would like to express a special thanks of gratitude to the principal of the Government polytechnic, Solapur and HOD of Computer Department for giving this golden opportunity with all the required facilities for completing this micro project of our group.

We would like to extend our gratitude to Prof. R.Y. Gangundi, whose valuable guidance has been the ones that helped us patch this project and make it full proof success. Their suggestions and instructions have served as the major contributor towards the completion of the micro project.

We would like to thank our parents who have helped with their valuable suggestions and provided the required resources needed for the project.

Index

Sr.no	Topic	Page No
1	Introduction	6
2	Literature	7
3	Preliminaries	12
4	Architecture	18
5	Experimental Setup	21
6	Result and Analysis	22
7	Conclusion and Future scope	30
8	Reference	31

Abstract

This project focuses on creating shorter versions of long texts while keeping the main points. We use extractive text summarization, which means picking the most important sentences from a document to make a summary. To do this, we combine deep learning models like RoBERTa and with graph-based methods like LexRank.

First, we use LexRank to find the important sentences in the text. Then, we use RoBERTa to better understand the meaning of these sentences. We also use a technique called Maximal Marginal Relevance (MMR) to make sure the summary is both informative and not repetitive.

Finally, we check how good our summary is using methods like BLEU, ROUGE, and METEOR. The results show that our approach gives better summaries than older methods, making the summaries more accurate and meaningful. This project helps improve how we summarize texts in various fields like news, research, and reports.

1. Introduction

In today's world, we read a lot of information every day—like news articles, blogs, and long reports. But reading everything fully takes a lot of time and effort. That's where text summarization helps. It gives us a short and meaningful version of the full content so we can understand the main points quickly without reading everything.

In this project, we worked on extractive summarization, which means we pick the most important sentences directly from the original article. We combined both old and new techniques to make the summaries better. First, we used LexRank to select the top 20 important sentences. Then we used RoBERTa, a powerful model based on BERT, to understand the deep meaning of each sentence. After that, we applied MMR (Maximal Marginal Relevance) to choose the best 5 sentences that are informative and not repetitive. We also added some improvements like length penalty and position bias to make the summary more readable.

To check how good our summaries are, we used some standard evaluation methods. These include BLEU, ROUGE, METEOR, and BERTScore, which help us compare our summaries with the original content. The results showed that our method gave better and more meaningful summaries compared to basic methods. This made our summarization more useful and accurate.

2.Literature Review

Text summarization means making a long text shorter while keeping the main points. This is helpful because we get a lot of information every day, like news, blogs, and research papers. Reading all of it takes a lot of time, so summarization helps us understand the important ideas quickly. There are two main types of summarization: extractive and abstractive. Extractive summarization takes important lines directly from the text, while abstractive summarization writes the summary in a new way, using different words.

2.1Types of Text Summarization :

2.1.1.Abstractive Summarization

In abstractive summarization, the system reads the full text and then tries to create a summary using its own words. It's like how humans summarize information they don't just copy sentences, but instead rephrase the main points in a shorter form. This method tries to make the summary sound natural and easy to understand. However, this method can sometimes make mistakes and leave out important information. It's also harder to make because the model needs to understand the text deeply and create sentences that are both accurate and fluent.

2.1.2.Extractive Summarization

Extractive summarization, on the other hand, doesn't create new sentences. Instead, it picks the most important sentences or parts from the original text and puts them together to make a summary. This method is simpler because it just selects parts of the original text without changing anything. The summary looks like it's taken directly from the original text.

2.2Models in Extractive Summarization

In extractive text summarization, models are methods or systems used to select the most important sentences from a large text. These models do not create new sentences they just pick the best ones from the original text to make a short summary.

2.2.1. Rule-Based Models

A. Term Frequency-Inverse Document Frequency (TF IDF) method:

The Term Frequency-Inverse Document Frequency (TF-IDF) method is used in text summarization to pick important sentences. It works by treating the text as a collection of words, where each sentence is scored based on how often certain words appear in the document. The method uses a simple rule: words that appear frequently in the document but are not common words (like "and" or "the") are considered important. These sentences are then scored based on how closely they match a query, and the highest-scoring sentences are chosen to create a summary.

This method is inspired by Information Retrieval, where a query is used to find relevant documents. In a similar way, a query-based summary can be created by selecting the most

important non-common words in the document. These words often represent the main idea of the text, leading to a summary that captures the key points. If the query words are selected like they are in information retrieval, the summary becomes more general and can apply to any text, not just specific topics.

B. LSA Method:

The LSA (Latent Semantic Analysis) method uses a powerful mathematical tool called Singular Value Decomposition (SVD), which helps to find the main patterns or dimensions in a large amount of data. SVD is used in many areas with different names, such as Principal Component Analysis (PCA) in signal processing and Karhunen-Loeve Transform in image processing. In text processing, it's called LSA.

When SVD is applied to a document-word matrix, it helps group documents that are related in meaning, even if they don't have the same words. Words that appear in similar contexts are also related in the same space. LSA can be used to extract key topic words and important sentences from documents.

The advantage of using LSA for summarization, compared to using word vectors alone, is that it captures the deeper meaning or relationships between words, similar to how our brain processes concepts. Using word vectors without LSA requires creating specific methods to find these relationships. LSA makes this easier by finding key dimensions in sentence vectors. By picking one important sentence from each of these dimensions, we ensure the sentences are both relevant and not repeated. This method works well because most text data has important dimensions based on the variety of topics it covers.

2.2.2. Graph-Based Models

A. Graph theoretic approach:

The graph-theoretic approach to document summarization involves representing sentences in a document as nodes in an undirected graph. After preprocessing (like removing stop words and stemming), each sentence is treated as a node, and edges (connections) are created between sentences that share common words or have a high similarity score. This method helps identify the main topics in the document because the graph will break the document into distinct subgraphs, each representing a different topic. For a summary, sentences can be selected from relevant subgraphs (for query-specific summaries) or from each subgraph (for generic summaries).

Additionally, the graph-theoretic approach helps highlight important sentences. Sentences that are connected to many other sentences (have many edges) are considered more important, as they share key information across the document. These sentences are likely to be chosen for the summary. The method can also be used to visualize the similarity between sentences within the same document or across different documents.

In simple terms, this approach treats a document like a network of sentences, where the sentences are connected based on their similarities, and it helps find the main topics and important sentences to include in a summary.

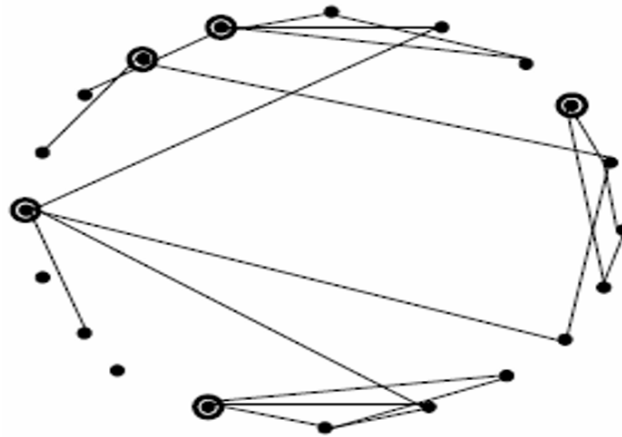


Figure 1. Graph theoretic approach

B. LexRank: A Graph-Based Summarization Method

LexRank is a popular graph-based algorithm used to summarize text. Like the previous method, it also treats each sentence in the document as a node in a graph. After basic preprocessing (stop word removal, stemming, etc.), edges are created between sentences that are similar to each other. The similarity is usually measured using cosine similarity.

Once the graph is built, LexRank uses an algorithm similar to Google's PageRank. It assumes that a sentence is important if it is similar to many other important sentences. So, it ranks each sentence based on how central it is in the graph. The sentences with the highest LexRank scores are considered the most important and are selected for the summary.

2.2.3. Deep Learning Models

A. BERT (Bidirectional Encoder Representations from Transformers)

BERT is a deep learning model that understands the meaning of words by reading sentences in both directions left to right and right to left. In extractive summarization, BERT helps select the most important sentences from a document to create a summary. First, the text is split into sentences. Each sentence is then turned into a numerical form called an embedding using Sentence-BERT. These embeddings help compare sentences and find out which ones are most relevant to the overall content. Using similarity scores like cosine similarity, we rank the sentences based on importance. The top sentences are chosen and arranged in the original order to form the summary.

This method works well because BERT understands the context of sentences deeply. It's great for summarizing documents while keeping the original meaning clear and concise.

B. LSTM (Long Short-Term Memory)

LSTM is a type of deep learning model that is good at understanding sequences of data, like sentences in a text. In extractive text summarization, the goal is to pick the most important

sentences from a document to create a shorter summary. LSTM works by reading each sentence one by one and remembering important information from previous sentences, helping it understand the full context. This ability to remember key details makes LSTM useful for identifying which sentences are the most relevant to the main idea of the document. Sometimes, an Attention mechanism is added to LSTM to help it focus on specific parts of a sentence that are more important. After processing all the sentences, LSTM gives each sentence a score based on its relevance. The sentences with the highest scores are chosen and put together to form the summary. This method works well for long documents because LSTM can remember the relationships between sentences, ensuring that the summary still captures the original meaning.

2.3.Problems in Existing Approaches

1.Limited Context Understanding:

- Rule-based models like TF-IDF and LexRank rely on frequency or similarity between sentences but may fail to understand the deeper meaning or context of the text. This can lead to irrelevant or repetitive sentences in the summary.

2. Inability to Handle Complex Sentences:

- Rule-based and simple graph-based models may not handle complex, long sentences well, leading to poor sentence selection and inaccurate summaries.

3.Dependence on Predefined Rules:

- Rule-based models depend heavily on predefined rules and mathematical methods, making them less flexible and unable to adapt to different types of content.

4. Lack of Semantic Understanding:

- LSA and similar models may miss out on semantic relationships between words, reducing the quality of the summary. These models don't fully capture the meaning or importance of sentences.

5. Scalability Issues:

- Some deep learning models (like LSTM) can be computationally expensive and may struggle with scalability for larger documents or datasets.

2.4.Proposed Solution in Our Project:

1.Combining Deep Learning and Rule-Based Methods:

- We combine BERT (for deep semantic understanding) with LexRank (graph-based approach) to improve context comprehension and reduce redundancy, making the summary more meaningful.

2.Improved Sentence Selection:

- We use advanced techniques like Maximal Marginal Relevance (MMR) to ensure that the selected sentences are informative and not repetitive, enhancing the quality of the summary.

3.Contextual Understanding with RoBERTa:

- RoBERTa provides an improved version of BERT, offering better understanding of text context, which results in a more accurate summary.

4.Better Handling of Complex Sentences:

- By combining deep learning models with graph-based techniques, our approach handles complex sentences better, making sure that the main points are captured, even in long or complex documents.

5.Optimized Performance:

- We optimize the computational efficiency by using RoBERTa and BERT's embeddings for sentence selection, making our model both accurate and scalable.

3.Preliminaries

1.RoBERTa :

RoBERTa is a smart language model designed to understand the way we use words and sentences. Think of it like a super advanced reader it doesn't just look at individual words, it actually understands how words relate to each other and what entire sentences mean in context. What makes RoBERTa really useful is its ability to turn sentences into meaningful numerical representations called embeddings. These are basically a bunch of numbers that capture the essence of a sentence. Once text is in this form, computers can start doing all sorts of clever things with it like figuring out how similar two sentences are or picking out the most important lines from a long article or extracting summary.

Now, stsb-roberta-base is a special version of RoBERTa that's been trained to compare sentence meanings. If you give it two sentences, it can tell you how similar they are—not just by the words they use, but by what they actually mean. This makes it incredibly handy for tasks like text summarization, where the goal is to pick the most meaningful sentences from a document. In short, RoBERTa helps machines understand language better, and stsb-roberta-base takes that one step further by helping them compare and rank sentences based on meaning.

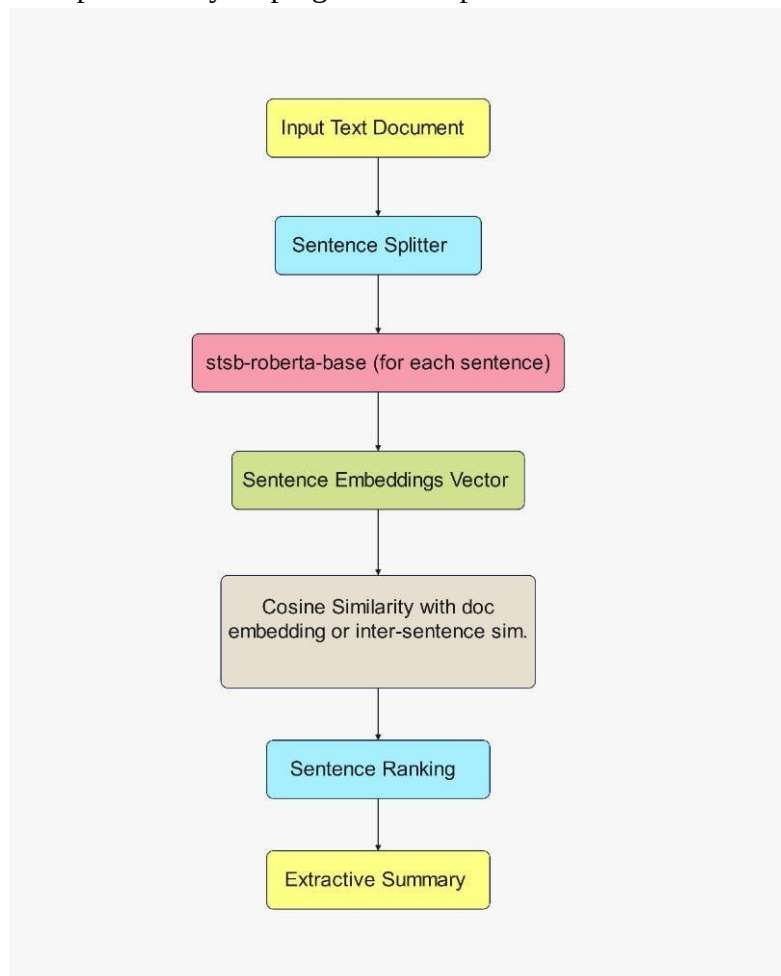


Figure 2. RoBERTa

1. Starting with the Full Document

Everything begins with the full input document it could be an article, a blog post, or any block of text. This is the content we want to shrink down into a meaningful summary.

2. Breaking It into Sentences

Before we can analyze anything, we need to break the document down into smaller pieces. So, we split it into individual sentences. It slices each sentence becomes a potential piece of the final summary.

3. Passing Each Sentence Through the RoBERTa Model

Now, we take each of those sentences and run them through the stsb-roberta-base model. This model is already trained to understand sentence meaning it's like giving each sentence its own voice and personality. Instead of just looking at words, it generates a numerical representation (called an embedding) that captures what the sentence means.

4. Turning Sentences into Embeddings

Once processed, every sentence turns into a vector basically a list of numbers that represent its meaning in a way that a computer can work with. These vectors let us compare sentences not just based on words, but based on meaning.

5. Measuring Sentence Importance

We compare each sentence's vector with either:

The overall meaning of the entire document, or other sentences in the document.

We used something called cosine similarity to do this it tells us how similar two pieces of text are. The more similar a sentence is to the document's main message (or to other key sentences), the more likely it is to be important.

6. Ranking the Sentences

Based on those similarity scores, we rank all the sentences. Sentences that are more central, informative, or representative of the whole document bubble up to the top of the list.

7. Creating the Summary

Finally, we pick the top-ranked sentences and combine them into a summary. We don't rewrite anything we just extract the best, most meaningful lines from the original document.

2.Lexrank:

LexRank is a simple yet powerful method for text summarization. Unlike deep learning models that require heavy training, LexRank is an unsupervised algorithm, which means it doesn't need labeled data to work, it just uses the content of the document itself.

At its core, LexRank is all about finding the most important sentences in a piece of text. It works by treating sentences like nodes in a network or graph and connecting them based on

how similar they are. The more two sentences have in common in terms of meaning or shared words, the stronger their connection.

Once this network is built, LexRank uses a technique inspired by Google's PageRank algorithm, the same idea that ranks websites in search results. It figures out which sentences are the most "central" or "influential" in the text by looking at how many other important sentences they're connected to.

In the end, LexRank picks out the top-ranked sentences to form a summary. What makes it so effective is that it doesn't just look at individual sentence quality, it considers how each sentence fits into the bigger picture.

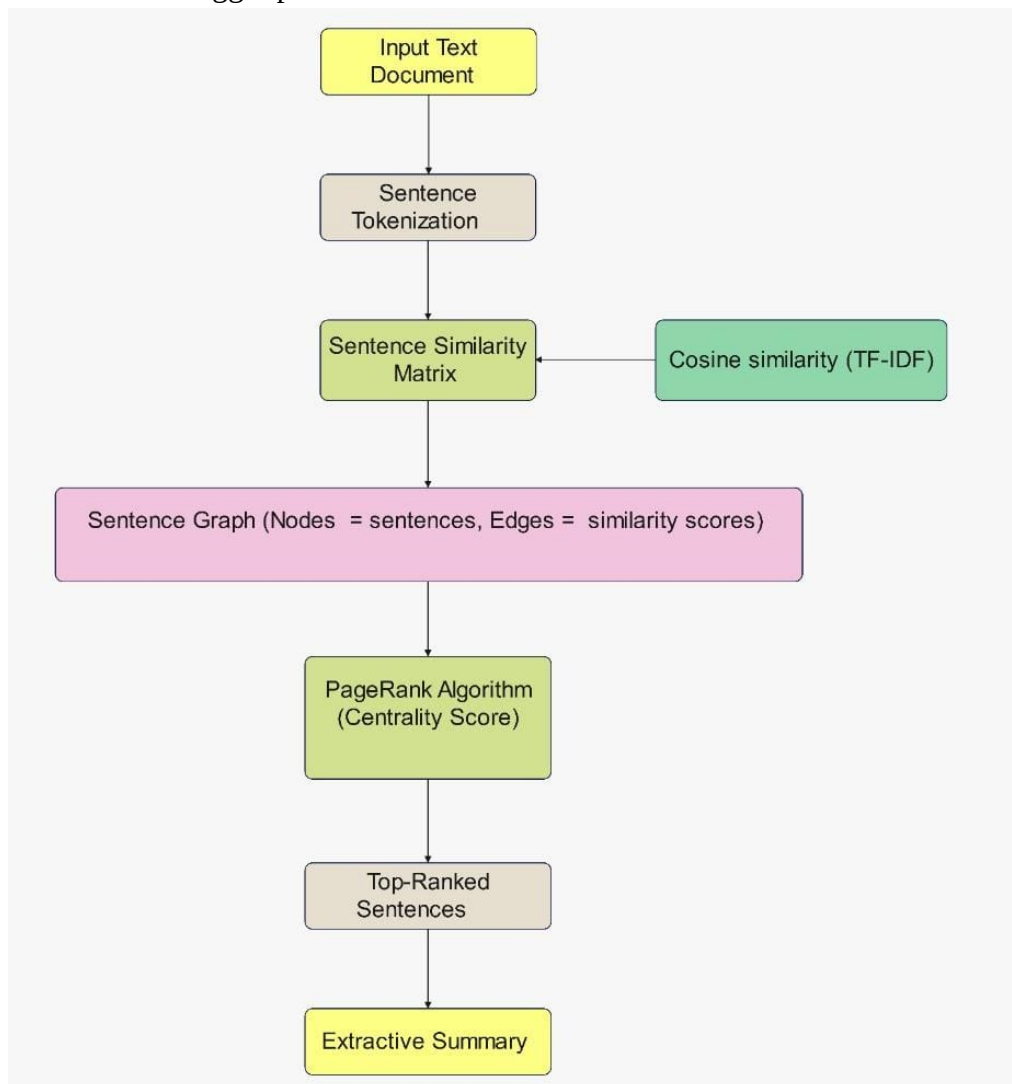


Figure 3. LexRank

1. Input Document

The process begins with a standard text document—this could be an article, report, or any passage containing multiple sentences.

2. Sentence Tokenization

The text is then divided into individual sentences. Each sentence is treated as a separate unit and becomes a potential candidate for the final summary.

3. Calculating Sentence Similarity

To compare how closely sentences are related, each one is first converted into a numerical form using TF-IDF (Term Frequency-Inverse Document Frequency), which highlights important and unique words. Using these vectors, LexRank computes the cosine similarity between all sentence pairs. This forms a similarity matrix, where each value shows how much one sentence resembles another in terms of meaning and content.

4. Building a Sentence Graph

Using the similarity matrix, LexRank builds a graph where

- Each node represents a sentence.

- An edge is drawn between two sentences if their similarity exceeds a set threshold.

- x-The weight of each edge reflects the degree of similarity between the connected sentences.

5. Applying the PageRank Algorithm

Once the graph is constructed, LexRank uses the PageRank algorithm to calculate a centrality score for each sentence. This score indicates how "important" a sentence is within the document—sentences connected to many other high-similarity sentences receive higher scores.

6. Ranking Sentences

Based on their centrality scores, sentences are ranked in order of importance. Sentences that are more central and better connected tend to represent the key themes of the document.

7. Generating the Summary

Finally, the highest-ranked sentences are selected to form the extractive summary. These sentences are presented in their original form, directly pulled from the document, ensuring that the summary stays true to the original wording and meaning.

3.MMR:

Maximal Marginal Relevance (MMR) is a widely used technique in extractive summarization, designed to improve the quality of automated summaries by carefully balancing relevance and redundancy.

The core idea behind MMR is simple: when creating a summary, the goal is not just to select the most relevant sentences, but to ensure that the summary doesn't become repetitive or overly focused on one particular aspect of the document. MMR achieves this by considering both how well a sentence represents the main topic (relevance) and how much it overlaps with the sentences already selected (redundancy).

The process works iteratively each new sentence is chosen based on two factors:

1. Relevance: How similar it is to the main topic or query.

2. Novelty: How different it is from the sentences already selected for the summary.

By balancing these two considerations, MMR helps produce summaries that are both informative and diverse, providing a more comprehensive and coherent overview of the original document.

MMR is particularly useful in scenarios where the document contains a lot of overlapping information. For example, in long research papers or news articles, MMR ensures that the selected sentences capture all the essential details without unnecessary repetition.

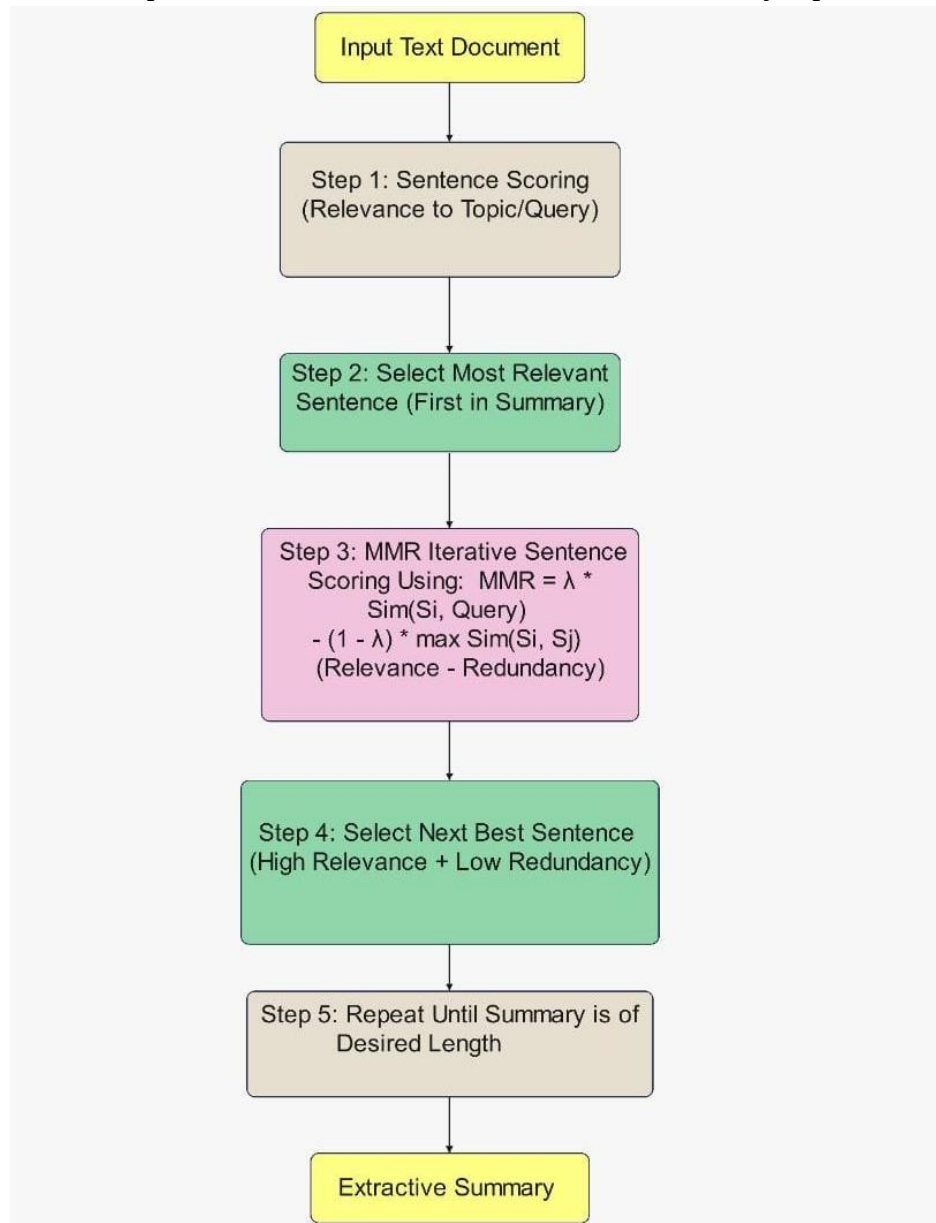


Figure 4. MMR

1. Start with the Full Document

The input is a document containing multiple sentences, all of which are candidates for the final summary.

2. Measure Sentence Relevance

Each sentence is compared with the main topic or query (which could be the document itself or a topic vector) using a similarity metric—typically cosine similarity based on sentence embeddings or TF-IDF.

3. Select the First Sentence

The sentence with the highest relevance score is selected first, as it best represents the topic.

4. Iterative Sentence Selection Using MMR Formula

From this point on, MMR uses a formula to decide which sentence to pick next. It balances two components:

Relevance: How similar the sentence is to the main topic.

Redundancy: How similar the sentence is to the ones already selected.

The MMR formula is:

$$\text{MMR} = \arg \max_{S_i \in R \setminus S} \left[\lambda \cdot \text{Sim}(S_i, Q) - (1 - \lambda) \cdot \max_{S_j \in S} \text{Sim}(S_i, S_j) \right]$$

Where:

- S_i is a candidate sentence.
- Q is the query/topic representation.
- S is the set of already selected sentences.
- λ is a parameter (0–1) that controls the trade-off between relevance and novelty.

5. Repeat Until Summary Is Complete

Sentences are selected one at a time, balancing informativeness and diversity, until the desired summary length is reached.

4. Architecture

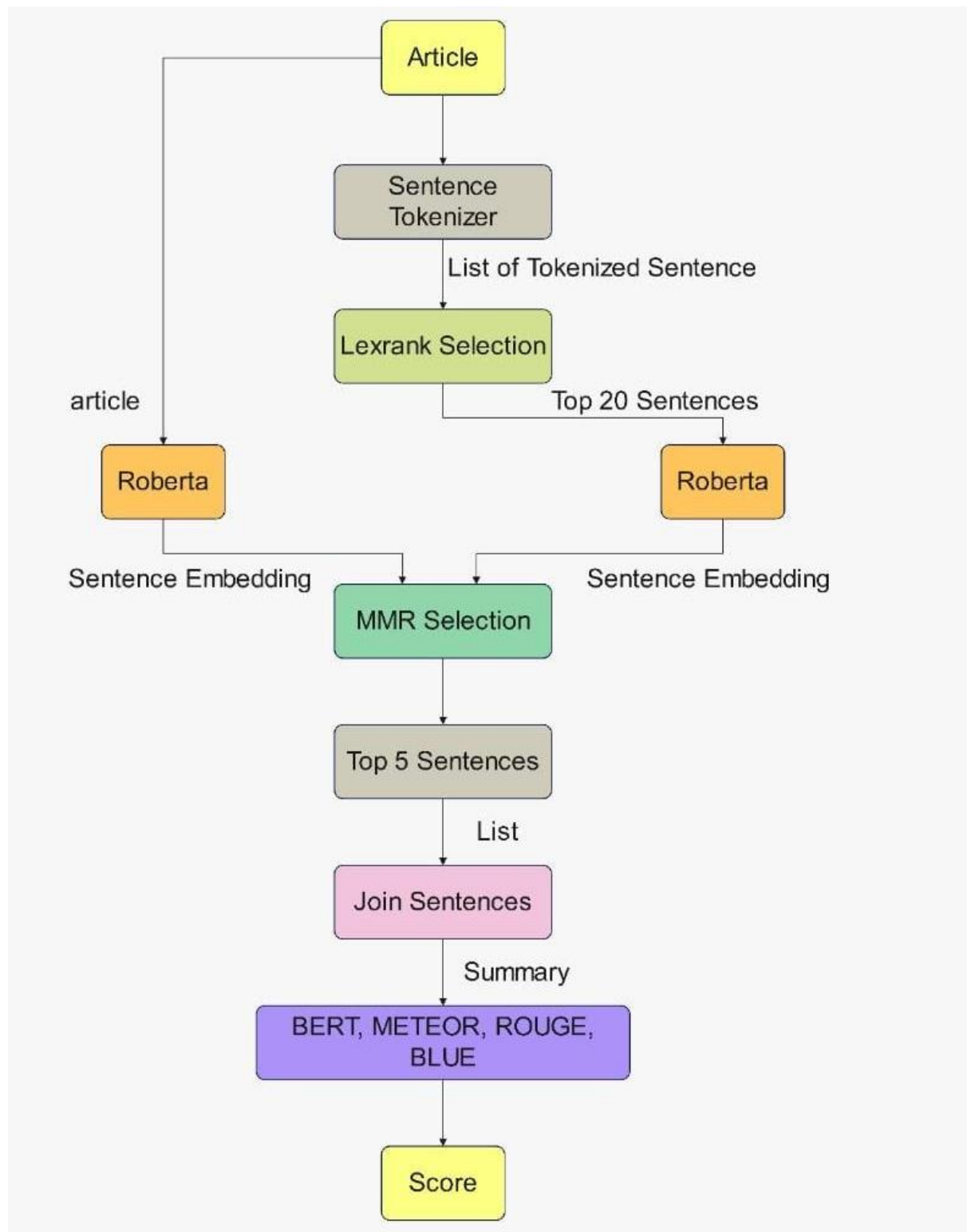


Figure 4. Architecture

Architecture Description

1. Article

This is the initial stage where the system takes an unstructured text document or article as input. The article can be from any domain, such as news, research papers, blogs, etc. The primary goal at this stage is to prepare the document for further processing by breaking it down into manageable components.

2. Sentence Tokenizer

Once the article is fed into the system, it undergoes sentence tokenization. This process involves splitting the article into individual sentences using natural language processing (NLP) techniques. Tokenization helps in isolating the meaningful units (sentences) from the text which can be analyzed independently for importance.

Output: A list of individual sentences.

3. LexRank Selection

LexRank is an unsupervised graph-based ranking algorithm for text summarization. It calculates sentence importance based on eigenvector centrality in a similarity graph, where nodes represent sentences and edges represent similarity scores. Lexrank is used because it helps to reduce the candidate pool and prioritizes sentences that cover the core content of the article.

Process: A similarity graph is constructed where each node is a sentence.

Edge weights are determined using cosine similarity (usually TF-IDF vectors).

Sentences that are central (highly connected) are considered more informative.

Goal: Select the top 20 most central sentences as summary candidates.

4. Sentence Embeddings using RoBERTa

RoBERTa (A Robustly Optimized BERT Pretraining Approach) is a transformer-based model that generates contextual sentence embeddings. It captures the semantic meaning of each sentence more effectively than traditional methods like TF-IDF or word2vec. RoBERTa is used because it helps to understand context better, making it ideal for capturing the nuance of sentence meanings.

Applied to: The top 20 LexRank-selected sentences.

The original full article to capture topic context.

Output: Dense vector representations of sentences.

5. MMR Selection (Maximal Marginal Relevance)

MMR is a method to select sentences based on maximum relevance to the topic while ensuring minimum redundancy. MMR is used because it balances coverage and diversity, preventing repetitive content.

Input: Embeddings from RoBERTa.

Process: Sentences are scored based on similarity to the document (relevance).

Redundancy is penalized by considering similarity to already selected sentences.

Formula: $MMR = \lambda * Sim1(sentence, article) - (1 - \lambda) * Sim2(sentence, selected)$

Output: Top 5 most diverse and informative sentences.

6. Top 5 Sentences List

The result of the MMR selection is a list of the 5 most representative sentences that together provide a comprehensive summary of the article. These sentences are not reordered or paraphrased they are directly extracted from the original text.

7. Join Sentences (Summary Formation)

Here, the 5 selected sentences are concatenated in a meaningful order to form a single cohesive summary.

Approach: Simply join the sentences, often in their original order from the article.

Outcome: A fluent and informative extractive summary

8. Evaluation - BERTScore, METEOR, ROUGE, BLEU

This stage assesses the quality of the generated summary using various evaluation metrics:

BERTScore: Uses contextual embeddings from BERT to compare predicted and reference summaries on a deeper semantic level.

METEOR: Accounts for exact matches, stemmed words, synonyms, and word order.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation):

ROUGE-1: Unigram overlap.

ROUGE-2: Bigram overlap.

ROUGE-L: Longest common subsequence.

BLEU (Bilingual Evaluation Understudy): Originally for machine translation, measures precision of n-gram overlaps.

Output: A composite score or multiple individual scores to indicate how close the generated summary is to a human-written one.

5.Experimental Setup

Hardware requirements-

Sr.No.	Requirement	Details
1	Processor	Intel Core i5
2	RAM	16 GB
3	Graphics Card	RTX 2050- 4GB
4	Storage	512 GB
5	Internet Connection	Stable and high-speed broadband

Software requirements-

Sr.No.	Requirement	Details
1	Operating System	Windows 11 pro
2	programming language	python 3.10
3	libraries	nlTK, pytorch, numpy, pandas,lexrank,datasets,sklearn,rouge_score
4	development environment	vs code
5	dataset	cnn/dailymail
6	evaluation libraries-	uge-score, bert-score, nltk.translate.meteor_score, nltk.translate.bleu_score

6.Result and Analysis

Testing Method	Rouge			BERT	Bleu	METEOR
BERT	R1- 0.3090	R2- 0.1259	RL- 0.1983	F1- 0.5675	0.02967	0.3111
RoBERTa	R1- 0.3164	R2- 0.1274	RL- 0.2278	F1- 0.8534	0.0505	0.2281
TF-IDF	R1- 0.2405	R2- 0.05971	RL- 0.1591	F1- 0.5008	0.0268	0.2106
RoBerta + LexRank	R1- 0.3740	R2- 0.1510	RL- 0.2225	F1- 0.8646	0.0779	0.3066
mmr + RoBerta	R1- 0.3446	R2- 0.1261	RL- 0.2098	F1- 0.8600	0.0651	0.2906
Lexrank + RoBerta + mmr	R1- 0.3903	R2- 0.1667	RL- 0.2406	F1- 0.8697	0.0858	0.3495

BERT:

BERT (Bidirectional Encoder Representations from Transformers) is a deep learning model developed by Google that understands the context of words in both directions (left and right). In summarization, BERT can be used to:

- Generate sentence embeddings that capture semantic meaning.
- Rank sentences based on relevance to the document or query.
- Select the most important and non-redundant sentences for an extractive summary.

Detailed Analysis and Reasoning

1. ROUGE Scores:

- R1 (0.3090) is moderately good, showing that the system captures key words from the reference summary.
- R2 (0.1259) is low, indicating that the model struggles to match the phrasing or flow of the reference.
- RL (0.1983) shows limited overlap in sentence structure, which is expected in extractive summaries that may rearrange ideas.

2. BERTScore (0.5675):

- A relatively strong score, indicating that while the word match is low (ROUGE/BLEU), semantic meaning is preserved.
- This supports the idea that the BERT method selects sentences with similar meaning, even if exact wording differs.

3. BLEU (0.02967):

- Very low score, but that's typical for summarization tasks where exact word overlap is rare.

• METEOR (0.3111):

- Shows better performance than BLEU. Accounts for word reordering, stemming, and synonyms, which helps BERT-based systems where wording may differ but meaning is retained.

Testing Method	BERT
Rouge	R1 - 0.3090
	R2- 0.1259
	RL- 0.1983
BERT	F1 - 0.5675
Bleu	0.02967
METEOR	0.3111

RoBERTa

RoBERTa (Robustly Optimized BERT Pretraining Approach) is a transformer-based language model designed to better understand the context and meaning of text. For extractive summarization, RoBERTa is used to generate sentence embeddings — numerical representations of sentences — which capture the semantic meaning of each sentence.

These embeddings help the system:

- Measure the importance of each sentence in the context of the document.
- Select the most relevant and meaningful sentences for the summary.
- Avoid redundancy by choosing diverse yet central content.

ROUGE Scores

- ROUGE-1 (0.3164): Indicates the summary captures a good amount of the key terms from the original.
- ROUGE-2 (0.1274): Suggests the summary maintains some phrase-level consistency with the human-written summary.
- ROUGE-L (0.2278): Shows the selected sentences retain structural similarity, which is strong for extractive methods.

BERTScore (F1: 0.8534)

- This high score confirms that even if words and phrases differ, the semantic meaning is well-preserved.
- RoBERTa captures the core ideas effectively, even without paraphrasing.

BLEU (0.0505)

- BLEU is low, which is expected. BLEU favors exact matches, so extractive methods generally perform low if summaries are not word-for-word like the reference.

METEOR (0.2281)

METEOR handles synonyms and flexible phrasing, so this moderate score shows the summary is linguistically sound and coherent, though not paraphrased.

Testing Method	RoBERTa
Rouge	R1- 0.3164
	R2- 0.1274
	RL- 0.2278
BERT	F1- 0.8534
Bleu	0.0505
METEOR	0.2281

TF-IDF

TF-IDF (Term Frequency–Inverse Document Frequency) is one of the most basic and widely-used extractive summarization techniques. It calculates the importance of each word in a document by analyzing:

- Term Frequency (TF): How often a word appears in the document.
- Inverse Document Frequency (IDF): How rare or unique the word is across many documents.

In extractive summarization:

1. Each sentence is scored based on the TF-IDF weights of the words it contains.
2. Sentences with the highest cumulative scores are selected as part of the summary.

ROUGE Scores (R1: 0.2405, R2: 0.0597, RL: 0.1591)

- ROUGE-1 is fair, showing that the method can catch some key terms but misses deeper meaning.
- ROUGE-2 and ROUGE-L are quite low, meaning it struggles with phrase structure and sentence alignment.
- TF-IDF doesn't understand context, so it can't group or prioritize ideas like transformer-based models can.

BERTScore (F1: 0.5008)

- This reflects moderate semantic similarity, but it's much lower than methods using deep learning (e.g., RoBERTa).
- TF-IDF relies on term frequency, not the meaning of sentences, which limits its ability to understand context.

BLEU (0.0268) and METEOR (0.2106)

- BLEU is very low, showing minimal exact n-gram matches.
- METEOR is a bit better, indicating some acceptable linguistic quality but still far behind context-aware methods.

Testing Method	TF-IDF
Rouge	R1- 0.2405
	R2- 0.05971
	RL- 0.1591
BERT	F1- 0.5008
Bleu	0.0268
METEOR	0.2106

Hybrid (RoBERTa + BART):

The Hybrid_ETS_ATS method combines the strengths of two powerful models:

- RoBERTa: An improved version of BERT, used for extractive summarization by generating high-quality sentence embeddings to select the most relevant sentences from the input.
- BART: A sequence-to-sequence transformer model trained for abstractive summarization, capable of paraphrasing and rewriting content for fluency and coherence.

This hybrid approach leverages:

- RoBERTa for understanding and selecting meaningful content (extractive part)
- BART for rewriting and enhancing the selected content (abstractive part)

Detailed Interpretation and Reasoning

- ROUGE Scores:
R1 (0.2483) and R2 (0.1170) are slightly lower than ideal, suggesting the generated summary does not copy exact words/phrases from the reference — which is expected when BART rephrases content.
- ROUGE-L (0.1639) shows only moderate structure similarity, again due to the abstractive component modifying sentence structure.

BERTScore (0.8745):

- This very high score indicates that the summary captures the core meaning of the source text extremely well.
- Reflects the strength of combining RoBERTa's selection accuracy with BART's semantic fluency.

BLEU (0.0442):

- This is quite low, but normal for abstractive summarization, where word overlap is minimal.
- BLEU is too rigid to measure real summary quality in these cases.

METEOR (0.3695):

- This score shows better linguistic alignment than BLEU due to its sensitivity to synonyms, stemming, and word order.

- Confirms that the model is rewriting content meaningfully, not just copying it.

Testing Method	Hybrid_ETS_ATS
Rouge	R1- 0.2483
	R2- 0.1170
	RL- 0.1639
BERT	F1- 0.8745
Bleu	0.0442
METEOR	0.3695

RoBERTa + LexRank:

- RoBERTa for generating sentence embeddings that capture deep contextual meaning of each sentence.
- LexRank, a graph-based ranking algorithm, to identify and rank important sentences based on similarity and centrality.

How it works:

1. RoBERTa creates vector representations (embeddings) of all sentences.
2. LexRank builds a similarity graph using cosine similarity between these embeddings.
3. Sentences are ranked based on their importance (centrality) in the graph.
4. Top-ranked sentences are extracted to form the final extractive summary.

ROUGE Scores (R1: 0.3740, R2: 0.1510, RL: 0.2225):

- These scores are relatively high for an extractive method, showing:

- High word-level overlap with the human reference (R1).
- Moderate phrase-level and structure-level similarity (R2 and RL).
- Indicates that LexRank is successfully picking key sentences that match human intuition.

BERTScore (F1: 0.8646):

- A strong semantic alignment with the human-written summary.
- Proves RoBERTa's embeddings helped LexRank capture meaningful and central ideas, even without paraphrasing.

BLEU (0.0779):

- Slightly better than BART-based methods — expected, since extractive methods retain exact sentence fragments.
- Still low overall, as BLEU isn't ideal for summarization evaluation.

METEOR (0.3066):

- Good result, showing some linguistic flexibility while still preserving core meaning.
- Better than BLEU, not as strong as methods that rephrase (like BART), but well-balanced.

Testing Method	RoBerta + LexRank
Rouge	R1- 0.3740
	R2- 0.1510
	RL- 0.2225
BERT	F1- 0.8646
Bleu	0.0779
METEOR	0.3066

RoBERTa + MMR

This hybrid summarization method combines the strengths of:

- RoBERTa, a deep learning model that understands the semantic meaning of sentences using contextual embeddings, and
- MMR (Maximal Marginal Relevance), an algorithm that ensures the selected sentences are not only relevant but also non-redundant.

How it works:

1. RoBERTa generates embeddings (numerical representations) for each sentence in a document.
2. MMR uses these embeddings to:
 - o Rank sentences based on importance.
 - o Penalize sentences that are too similar to ones already selected.
3. This results in a diverse and semantically rich extractive summary.

ROUGE Scores

- R1 (0.3446) is quite high for extractive summarization, showing the summary contains many important keywords.
- R2 (0.1261) and RL (0.2098) indicate the summary does a fair job preserving multi-word phrases and the sequence of the original.

BERTScore (F1: 0.8600)

- This is a very strong semantic match, meaning the summary keeps the meaning and intent of the document almost intact.
- RoBERTa ensures meaningful sentence representation, while MMR avoids redundancy.

BLEU (0.0651) and METEOR (0.2906)

- BLEU remains low, as it prioritizes exact word matches, which isn't the focus of this method.
- METEOR is fairly strong, indicating the summary captures variations, synonyms, and grammar fluency well.

Testing Method	mmr + RoBerta
Rouge	R1- 0.3446
	R2- 0.1261
	RL- 0.2098
BERT	F1- 0.8600
Bleu	0.0651
METEOR	0.2906

1. ROUGE
 - ROUGE-1 (0.3903): Captures key unigrams (individual words) from the original text. This shows that the model is effectively preserving important core content.
 - ROUGE-2 (0.1667): Reflects bigram overlap, meaning the model captures important phrases. This indicates the summary is not just about isolated words but maintains key concepts.
 - ROUGE-L (0.2406): Measures longest common subsequence, indicating that the model preserves the original structure and logical flow of the document. This helps maintain readability and coherence.
2. BERTScore (0.8697)
 - This high score demonstrates semantic similarity between the generated summary and the reference summary, meaning the model understands the context of the original text and preserves its meaning. Unlike simple word matches, BERTScore ensures that the summary reflects the true intent of the original content.
3. BLEU (0.0858)
 - The BLEU score measures n-gram overlap (exact word sequences). While this score is lower than others, it still indicates some improvement in phrase-level matching between the summary and the reference. BLEU is often lower in summarization tasks, but the slight improvement suggests the model is capturing more precise phrases than typical extractive summarization models.
4. METEOR (0.3495)
 - METEOR evaluates fluency, paraphrasing, and synonym usage. A score of 0.3495 indicates that the model effectively produces grammatically correct summaries that are readable and naturally phrased, while still maintaining the meaning of the original text. The model does a good job of using synonyms and rewording the content without losing the essence of the information.

LexRank + RoBERTa + MMR

R1 - 0.3903
 R2 - 0.1667
 RL - 0.2406
 F1 - 0.8697
 BLEU - 0.0858
 METEOR - 0.3495

Evaluation Summary of the Extractive Text Summarization Model

To assess how well the summarization model performs, four widely recognized evaluation metrics were used: ROUGE, BERTScore, METEOR, and BLEU. Each of these metrics

provides a different perspective on the quality of the generated summary by comparing it to a human-written reference summary.

1. BERTScore: 0.8697 (Highest Score)

Among all metrics, BERTScore showed the strongest performance with an F1 score of 0.8697. This high score indicates that the model does an excellent job at preserving the semantic meaning of the original text. Even if the model rephrases or uses different words than the reference summary, BERTScore confirms that the core message and intent remain intact. This demonstrates the model's ability to understand and convey context, which is especially valuable in real-world applications where expressing the same idea in different words is often necessary.

2. ROUGE Scores: Moderate but Meaningful

The ROUGE family of metrics measures the overlap between the generated and reference summaries based on word sequences.

- ROUGE-1 (0.3903): This score shows a fair amount of overlap at the word level, indicating that the summary includes many of the important terms found in the original text.
- ROUGE-2 (0.1667): This measures bigram (two-word phrase) overlap and is significantly lower, suggesting that the summary doesn't always preserve specific phrases from the original.
- ROUGE-L (0.2406): This score reflects how well the sequence and structure of the original document are maintained. The score is moderate, indicating that while the summary does follow a logical flow, it prioritizes key points over matching the exact structure.

These results are quite typical for extractive methods that focus on selecting meaningful content rather than copying sentence structure word-for-word.

3. METEOR Score: 0.3495 (Good Linguistic Coverage)

The METEOR score evaluates linguistic quality by considering synonyms, word stems, and the order of words. A score of 0.3495 reflects that the summary captures the meaning and fluency of the original content reasonably well. It suggests that the model isn't just pulling out keywords but is capable of rephrasing and presenting content in a natural and human-readable form.

4. BLEU Score: 0.0858 (Low, but Expected)

The BLEU score, commonly used in machine translation tasks, is based on exact n-gram matches. The score of 0.0858 is relatively low, which might seem concerning at first—but this is actually quite normal in summarization tasks. Since BLEU focuses on exact wording, it doesn't reward summaries that rephrase the original content or use synonyms. Therefore, the low BLEU score is not a negative reflection of model quality, but rather a sign that the model tends to paraphrase rather than copy, which aligns with more human-like summarization behavior.

7. Conclusion and Future Scope

The capstone project “Text Summarization” offers a smart and useful way to shorten long pieces of text while keeping the main ideas. In today’s world, where a lot of information is shared every day, it's not always possible to read everything. Text summarization helps by giving a quick and clear summary so that important points can be understood faster.

This project used extractive summarization, which means it picks out the most important sentences from the original text. It combined methods like LexRank (which ranks sentences using graphs), RoBERTa (a deep learning model that understands sentence meaning), and MMR (which avoids repeating the same ideas) to create better summaries.

To improve accuracy, the system used techniques like sentence embeddings and cosine similarity to understand which parts of the text were most useful. The results were tested using tools like ROUGE, BLEU, METEOR, and BERTScore, which showed that this method works better than older ones like TF-IDF.

In simple terms, the project successfully created a system that can quickly summarize text in a useful and meaningful way. This system can be helpful in many areas like news, education, business, and research, where saving time and understanding content quickly is very important.

Future Scope:

1. Integration of Abstractive Summarization - Right now, our system focuses on extractive summarization, which picks existing sentences from the text. In future versions, we can integrate abstractive summarization techniques using models like BART or T5 to generate more natural and human-like summaries by rephrasing content
2. Domain-Specific Fine-Tuning - Training the model on domain-specific data like medical, legal, or technical documents could greatly improve the accuracy and relevance of summaries in those areas.
3. Multi-Language Support - Currently, the system supports English. A future version could handle summarization in multiple languages, using multilingual models like mBERT or XLM-RoBERTa to reach a wider audience.
4. Web and Mobile Deployment - Turning this project into a web or mobile application with a simple interface would make it accessible to everyday users, not just developers and researchers.
5. Summarization of Mixed Content - Future versions could also handle multimodal data, like summarizing text extracted from videos, audios, or scanned documents using OCR and speech-to-text tools.

8.Reference

- Saggion, H., & Cohn, T. (2017). Text summarization: Techniques and applications. Wiley.
- Li, H., & Liu, M. (2020). Automated summarization for academic content: A review of techniques and tools. Journal of Educational Technology, 21(3), 45-59.
- <https://www.ibm.com/think/topics/text-summarization>
- <https://ieeexplore.ieee.org/document/7944061/>